# Prompting best practices

Copy page

This guide provides specific prompt engineering techniques for Claude 4.x models, with specific guidance for Sonnet 4.5, Haiku 4.5, and Opus 4.5. These models have been trained for more precise instruction following than previous generations of Claude models.

For an overview of Claude 4.5's new capabilities, see What's new in Claude 4.5. For migration guidance from previous models, see Migrating to Claude 4.5.

## General principles

### Be explicit with your instructions

Claude 4.x models respond well to clear, explicit instructions. Being specific about your desired output can help enhance results. Customers who desire the "above and beyond" behavior from previous Claude models might need to more explicitly request these behaviors with newer models.

**Example: Creating an analytics dashboard**

### Add context to improve performance

Providing context or motivation behind your instructions, such as explaining to Claude why such behavior is important, can help Claude 4.x models better understand your goals and deliver more targeted responses.

**Example: Formatting preferences**

Claude is smart enough to generalize from the explanation.

Claude 4.x models pay close attention to details and examples as part of their precise instruction following capabilities. Ensure that your examples align with the behaviors you want to encourage and minimize behaviors you want to avoid.

## Long-horizon reasoning and state tracking

Claude 4.5 models excel at long-horizon reasoning tasks with exceptional state tracking capabilities. It maintains orientation across extended sessions by focusing on incremental progress—making steady advances on a few things at a time rather than attempting everything at once. This capability especially emerges over multiple context windows or task iterations, where Claude can work on a complex task, save the state, and continue with a fresh context window.

### Context awareness and multi-window workflows

Claude 4.5 models feature context awareness, enabling the model to track its remaining context window (i.e. "token budget") throughout a conversation. This enables Claude to execute tasks and manage context more effectively by understanding how much space it has to work.

**Managing context limits:**

If you are using Claude in an agent harness that compacts context or allows saving context to external files (like in Claude Code), we suggest adding this information to your prompt so Claude can behave accordingly. Otherwise, Claude may sometimes naturally try to wrap up work as it approaches the context limit. Below is an example prompt:

Sample prompt

```
Your context window will be automatically compacted as it approaches its limit, allowing
```

The memory tool pairs naturally with context awareness for seamless context transitions.

### Multi-context window workflows

For tasks spanning multiple context windows:

1. **Use a different prompt for the very first context window**: Use the first context window to set up a framework (write tests, create setup scripts), then use future context windows to iterate on a todo-list.
2. **Have the model write tests in a structured format**: Ask Claude to create tests before starting work and keep track of them in a structured format (e.g., `tests.json`). This leads to better long-

3. **Set up quality of life tools**: Encourage Claude to create setup scripts (e.g., `init.sh`) to gracefully start servers, run test suites, and linters. This prevents repeated work when continuing from a fresh context window.
4. **Starting fresh vs compacting**: When a context window is cleared, consider starting with a brand new context window rather than using compaction. Claude 4.5 models are extremely effective at discovering state from the local filesystem. In some cases, you may want to take advantage of this over compaction. Be prescriptive about how it should start:
   - "Call pwd; you can only read and write files in this directory."
   - "Review progress.txt, tests.json, and the git logs."
   - "Manually run through a fundamental integration test before moving on to implementing new features."
5. **Provide verification tools**: As the length of autonomous tasks grows, Claude needs to verify correctness without continuous human feedback. Tools like Playwright MCP server or computer use capabilities for testing UIs are helpful.
6. **Encourage complete usage of context**: Prompt Claude to efficiently complete components before moving on:

Sample prompt

```
This is a very long task, so it may be beneficial to plan out your work clearly. It's enc
```

## State management best practices

- **Use structured formats for state data**: When tracking structured information (like test results or task status), use JSON or other structured formats to help Claude understand schema requirements
- **Use unstructured text for progress notes**: Freeform progress notes work well for tracking general progress and context
- **Use git for state tracking**: Git provides a log of what's been done and checkpoints that can be restored. Claude 4.5 models perform especially well in using git to track state across multiple sessions.
- **Emphasize incremental progress**: Explicitly ask Claude to keep track of its progress and focus on incremental work

**Example: State tracking**

Claude 4.5 models have a more concise and natural communication style compared to previous models:

- **More direct and grounded**: Provides fact-based progress reports rather than self-celebratory updates
- **More conversational**: Slightly more fluent and colloquial, less machine-like
- **Less verbose**: May skip detailed summaries for efficiency unless prompted otherwise

This communication style accurately reflects what has been accomplished without unnecessary elaboration.

## Guidance for specific situations

### Balance verbosity

Claude 4.5 models tend toward efficiency and may skip verbal summaries after tool calls, jumping directly to the next action. While this creates a streamlined workflow, you may prefer more visibility into its reasoning process.

If you want Claude to provide updates as it works:

Sample prompt

```
After completing a task that involves tool use, provide a quick summary of the work you've
```

### Tool usage patterns

Claude 4.5 models are trained for precise instruction following and benefits from explicit direction to use specific tools. If you say "can you suggest some changes," it will sometimes provide suggestions rather than implementing them—even if making changes might be what you intended.

For Claude to take action, be more explicit:

> **Example: Explicit instructions**

To make Claude more proactive about taking action by default, you can add this to your system prompt:

```
<default_to_action>
By default, implement changes rather than only suggesting them. If the user's intent is u
</default_to_action>
```

On the other hand, if you want the model to be more hesitant by default, less prone to jumping straight into implementations, and only take action if requested, you can steer this behavior with a prompt like the below:

Sample prompt for conservative action

```
<do_not_act_before_instructions>
Do not jump into implementatation or changes files unless clearly instructed to make chan
</do_not_act_before_instructions>
```

## Tool usage and triggering

Claude Opus 4.5 is more responsive to the system prompt than previous models. If your prompts were designed to reduce undertriggering on tools or skills, Claude Opus 4.5 may now overtrigger. The fix is to dial back any aggressive language. Where you might have said "CRITICAL: You MUST use this tool when...", you can use more normal prompting like "Use this tool when...".

## Control the format of responses

There are a few ways that we have found to be particularly effective in steering output formatting in Claude 4.x models:

1. **Tell Claude what to do instead of what not to do**
   - Instead of: "Do not use markdown in your response"
   - Try: "Your response should be composed of smoothly flowing prose paragraphs."
2. **Use XML format indicators**
   - Try: "Write the prose sections of your response in <smoothly_flowing_prose_paragraphs> tags."
3. **Match your prompt style to the desired output**

   The formatting style used in your prompt may influence Claude's response style. If you are still experiencing steerability issues with output formatting, we recommend as best as you can

4. **Use detailed prompts for specific formatting preferences**

   For more control over markdown and formatting usage, provide explicit guidance:

Sample prompt to minimize markdown

```
<avoid_excessive_markdown_and_bullet_points>
When writing reports, documents, technical explanations, analyses, or any long-form conte

DO NOT use ordered lists (1. ...) or unordered lists (*) unless : a) you're presenting tr

Instead of listing items with bullets or numbers, incorporate them naturally into sentenc

Your goal is readable, flowing text that guides the reader naturally through ideas rather
</avoid_excessive_markdown_and_bullet_points>
```

## Research and information gathering

Claude 4.5 models demonstrate exceptional agentic search capabilities and can find and synthesize information from multiple sources effectively. For optimal research results:

1. **Provide clear success criteria**: Define what constitutes a successful answer to your research question
2. **Encourage source verification**: Ask Claude to verify information across multiple sources
3. **For complex research tasks, use a structured approach**:

Sample prompt for complex research

```
Search for this information in a structured way. As you gather data, develop several compe
```

This structured approach allows Claude to find and synthesize virtually any piece of information and iteratively critique its findings, no matter the size of the corpus.

## Subagent orchestration

Claude 4.5 models demonstrate significantly improved native subagent orchestration capabilities. These models can recognize when tasks would benefit from delegating work to specialized subagents and do so proactively without requiring explicit instruction.

To take advantage of this behavior:

2. **Let Claude orchestrate naturally**: Claude will delegate appropriately without explicit instruction
3. **Adjust conservativeness if needed**:

Sample prompt for conservative subagent usage

```
Only delegate to subagents when the task clearly benefits from a separate agent with a ne
```

## Model self-knowledge

If you would like Claude to identify itself correctly in your application or use specific API strings:

Sample prompt for model identity

```
The assistant is Claude, created by Anthropic. The current model is Claude Sonnet 4.5.
```

For LLM-powered apps that need to specify model strings:

Sample prompt for model string

```
When an LLM is needed, please default to Claude Sonnet 4.5 unless the user requests other
```

## Thinking sensitivity

When extended thinking is disabled, Claude Opus 4.5 is particularly sensitive to the word "think" and its variants. We recommend replacing "think" with alternative words that convey similar meaning, such as "consider," "believe," and "evaluate."

## Leverage thinking & interleaved thinking capabilities

Claude 4.x models offer thinking capabilities that can be especially helpful for tasks involving reflection after tool use or complex multi-step reasoning. You can guide its initial or interleaved thinking for better results.

Example prompt

For more information on thinking capabilities, see Extended thinking.

## Document creation

Claude 4.5 models excel at creating presentations, animations, and visual documents. These models match or exceed Claude Opus 4.1 in this domain, with impressive creative flair and stronger instruction following. The models produce polished, usable output on the first try in most cases.

For best results with document creation:

Sample prompt

```
Create a professional presentation on [topic]. Include thoughtful design elements, visual
```

## Improved vision capabilities

Claude Opus 4.5 has improved vision capabilities compared to previous Claude models. It performs better on image processing and data extraction tasks, particularly when there are multiple images present in context. These improvements carry over to computer use, where the model can more reliably interpret screenshots and UI elements. You can also use Claude Opus 4.5 to analyze videos by breaking them up into frames.

One technique we've found effective to further boost performance is to give Claude Opus 4.5 a crop tool or skill. We've seen consistent uplift on image evaluations when Claude is able to "zoom" in on relevant regions of an image. We've put together a cookbook for the crop tool here.

## Optimize parallel tool calling

Claude 4.x models excel at parallel tool execution, with Sonnet 4.5 being particularly aggressive in firing off multiple operations simultaneously. Claude 4.x models will:

- Run multiple speculative searches during research
- Read several files at once to build context faster
- Execute bash commands in parallel (which can even bottleneck system performance)

This behavior is easily steerable. While the model has a high success rate in parallel tool calling without prompting, you can boost this to ~100% or adjust the aggression level:

```
<use_parallel_tool_calls>
If you intend to call multiple tools and there are no dependencies between the tool calls
</use_parallel_tool_calls>
```

Sample prompt to reduce parallel execution

```
Execute operations sequentially with brief pauses between each step to ensure stability.
```

## Reduce file creation in agentic coding

Claude 4.x models may sometimes create new files for testing and iteration purposes, particularly when working with code. This approach allows Claude to use files, especially python scripts, as a 'temporary scratchpad' before saving its final output. Using temporary files can improve outcomes particularly for agentic coding use cases.

If you'd prefer to minimize net new file creation, you can instruct Claude to clean up after itself:

Sample prompt

```
If you create any temporary new files, scripts, or helper files for iteration, clean up tl
```

## Overeagerness and file creation

Claude Opus 4.5 has a tendency to overengineer by creating extra files, adding unnecessary abstractions, or building in flexibility that wasn't requested. If you're seeing this undesired behavior, add explicit prompting to keep solutions minimal.

For example:

Sample prompt to minimize overengineering

```
Avoid over-engineering. Only make changes that are directly requested or clearly necessary

Don't add features, refactor code, or make "improvements" beyond what was asked. A bug fix

Don't add error handling, fallbacks, or validation for scenarios that can't happen. Trust

Don't create helpers, utilities, or abstractions for one-time operations. Don't design for
```

Claude 4.x models, particularly Opus 4.5, excel at building complex, real-world web applications with strong frontend design. However, without guidance, models can default to generic patterns that create what users call the "AI slop" aesthetic. To create distinctive, creative frontends that surprise and delight:

> For a detailed guide on improving frontend design, see our blog post on improving frontend design through skills.

Here's a system prompt snippet you can use to encourage better frontend design:

Sample prompt for frontend aesthetics

```
<frontend_aesthetics>
You tend to converge toward generic, "on distribution" outputs. In frontend design, this (

Focus on:
- Typography: Choose fonts that are beautiful, unique, and interesting. Avoid generic font
- Color & Theme: Commit to a cohesive aesthetic. Use CSS variables for consistency. Domina
- Motion: Use animations for effects and micro-interactions. Prioritize CSS-only solutions
- Backgrounds: Create atmosphere and depth rather than defaulting to solid colors. Layer (

Avoid generic AI-generated aesthetics:
- Overused font families (Inter, Roboto, Arial, system fonts)
- Clichéd color schemes (particularly purple gradients on white backgrounds)
- Predictable layouts and component patterns
- Cookie-cutter design that lacks context-specific character

Interpret creatively and make unexpected choices that feel genuinely designed for the cont
</frontend_aesthetics>
```

You can also refer to the full skill here.

## Avoid focusing on passing tests and hard-coding

Claude 4.x models can sometimes focus too heavily on making tests pass at the expense of more general solutions, or may use workarounds like helper scripts for complex refactoring instead of using standard tools directly. To prevent this behavior and ensure robust, generalizable solutions:

Sample prompt

```
Focus on understanding the problem requirements and implementing the correct algorithm. T

If the task is unreasonable or infeasible, or if any of the tests are incorrect, please i
```

### Encouraging code exploration

Claude Opus 4.5 is highly capable but can be overly conservative when exploring code. If you notice the model proposing solutions without looking at the code or making assumptions about code it hasn't read, the best solution is to add explicit instructions to the prompt. Claude Opus 4.5 is our most steerable model to date and responds reliably to direct guidance.

For example:

Sample prompt for code exploration

```
ALWAYS read and understand relevant files before proposing code edits. Do not speculate a
```

### Minimizing hallucinations in agentic coding

Claude 4.x models are less prone to hallucinations and give more accurate, grounded, intelligent answers based on the code. To encourage this behavior even more and minimize hallucinations:

Sample prompt

```
<investigate_before_answering>
Never speculate about code you have not opened. If the user references a specific file, y
</investigate_before_answering>
```

## Migration considerations

When migrating to Claude 4.5 models:

1. **Be specific about desired behavior**: Consider describing exactly what you'd like to see in the output.
2. **Frame your instructions with modifiers**: Adding modifiers that encourage Claude to increase the quality and detail of its output can help better shape Claude's performance. For example, instead of "Create an analytics dashboard", use "Create an analytics dashboard. Include as many relevant

3. **Request specific features explicitly**: Animations and interactive elements should be requested explicitly when desired.